

Сетевой клиент

1.0

Generated by Doxygen 1.9.1

1	Создание	Создание демо файлов	1
1.1	Проверка	справки (высший приоритет)	1
1.2	В отдельном терминале	запустите сервер	1
1.3	Запуск	клиента	1
1.4	Запуск	тестов	1
1.5	Опции		1
2	Test	List	3
3	Class	Index	5
3.1	Class	List	5
4	File	Index	7
4.1	File	List	7
5	Class	Documentation	9
5.1	AuthManager	Class Reference	9
5.1.1	Detailed	Description	9
5.1.2	Constructor &	Destructor Documentation	9
5.1.2.1	AuthManager()		9
5.1.2.2	~AuthManager()		10
5.1.3	Member	Function Documentation	10
5.1.3.1	computeHash()		10
5.1.3.2	generateSalt()		10
5.2	ClientParams	Struct Reference	10
5.2.1	Detailed	Description	11
5.2.2	Member	Data Documentation	11
5.2.2.1	config_file		11
5.2.2.2	input_file		11
5.2.2.3	output_file		11
5.2.2.4	port		11
5.2.2.5	server_address		12
5.3	ConnectionManager	Class Reference	12
5.3.1	Detailed	Description	12
5.3.2	Constructor &	Destructor Documentation	12
5.3.2.1	ConnectionManager()		13
5.3.2.2	~ConnectionManager()		13
5.3.3	Member	Function Documentation	13
5.3.3.1	authenticate()		13
5.3.3.2	processVectors()		13
5.3.3.3	receiveData()		13
5.3.3.4	receiveString()		14
5.3.3.5	sendData()		14
5.3.3.6	sendString()		14
5.3.4	Member	Data Documentation	14

5.3.4.1	port_	14
5.3.4.2	server_addr_	14
5.3.4.3	socket_	15
5.4	FileHandler Class Reference	15
5.4.1	Detailed Description	15
5.4.2	Constructor & Destructor Documentation	15
5.4.2.1	FileHandler() [1/2]	15
5.4.2.2	~FileHandler()	16
5.4.2.3	FileHandler() [2/2]	16
5.4.3	Member Function Documentation	16
5.4.3.1	operator=()	16
5.4.3.2	readConfig()	16
5.4.3.3	readVectors()	16
5.4.3.4	saveResults()	17
5.5	TestFileFixture Struct Reference	17
5.5.1	Detailed Description	17
5.5.2	Constructor & Destructor Documentation	17
5.5.2.1	TestFileFixture()	18
5.5.2.2	~TestFileFixture()	18
5.5.3	Member Data Documentation	18
5.5.3.1	test_filename	18
5.5.3.2	test_output	19
5.6	ManagerInterface Class Reference	19
5.6.1	Detailed Description	20
5.6.2	Constructor & Destructor Documentation	20
5.6.2.1	ManagerInterface()	20
5.6.3	Member Function Documentation	20
5.6.3.1	getHelp()	20
5.6.3.2	getParams()	20
5.6.3.3	parseArguments()	20
5.6.4	Member Data Documentation	21
5.6.4.1	desc_	21
5.6.4.2	params_	21
6	File Documentation	23
6.1	README.md File Reference	23
6.2	src/auth_manager.cpp File Reference	23
6.2.1	Detailed Description	23
6.3	src/auth_manager.h File Reference	24
6.3.1	Detailed Description	24
6.3.2	Macro Definition Documentation	24
6.3.2.1	CRYPTOPP_ENABLE_NAMESPACE_WEAK	25
6.4	src/connection_manager.cpp File Reference	25

6.4.1 Detailed Description	25
6.5 src/connection_manager.h File Reference	25
6.5.1 Detailed Description	26
6.6 src/file_handler.cpp File Reference	26
6.6.1 Detailed Description	26
6.7 src/file_handler.h File Reference	27
6.7.1 Detailed Description	27
6.8 src/main.cpp File Reference	28
6.8.1 Detailed Description	28
6.8.2 Function Documentation	28
6.8.2.1 createConfigFile()	28
6.8.2.2 createDemoInputFile()	29
6.8.2.3 ensureFilesExist()	29
6.8.2.4 main()	29
6.9 src/test_main.cpp File Reference	30
6.9.1 Detailed Description	31
6.9.2 Function Documentation	31
6.9.2.1 main()	31
6.9.2.2 SUITE() [1/3]	32
6.9.2.3 SUITE() [2/3]	32
6.9.2.4 SUITE() [3/3]	32
6.10 src/user_interface.cpp File Reference	33
6.10.1 Detailed Description	33
6.11 src/user_interface.h File Reference	33
6.11.1 Detailed Description	34
7 Example Documentation	35
7.1 /home/stud/client_project/src/test_main.cpp	35

Глава 1

Создание Создание демо файлов

```
make demo
```

1.1 Проверка справки (высший приоритет)

```
./client -h
```

1.2 В отдельном терминале запустите сервер

```
./server_static -T int32_t -H MD5 -S client
```

1.3 Запуск клиента

```
./client -s 127.0.0.1 -i test_vectors.txt -o results.bin
```

1.4 Запуск тестов

```
make test
```

1.5 Опции

Опция Описание Значение по умолчанию -h Показать справку -s Адрес сервера - -p Порт сервера 33333 -i Входной файл - -o Выходной файл - -c Файл конфигурации с ~/.config/vclient.conf логином и паролем

Глава 2

Test List

Member **SUITE** (UserInterfaceTest)

HelpShort

Member **SUITE** (FileHandlerTest)

ReadVectorsValid

Member **SUITE** (AuthManagerTest)

GenerateSaltLength

Глава 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AuthManager	Статический класс для криптографических операций	9
ClientParams	Структура для хранения параметров клиента	10
ConnectionManager	Класс для управления TCP-соединением с сервером	12
FileHandler	Статический класс для операций с файлами	15
TestFileFixture	Фикстура для тестирования работы с файлами	17
UserInterface	Класс для обработки параметров командной строки	19

Глава 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

src/ auth_manager.cpp	
Реализация класса аутентификации	23
src/ auth_manager.h	
Заголовочный файл класса аутентификации	24
src/ connection_manager.cpp	
Реализация класса управления соединением	25
src/ connection_manager.h	
Заголовочный файл класса управления соединением	25
src/ file_handler.cpp	
Реализация класса работы с файлами	26
src/ file_handler.h	
Заголовочный файл класса работы с файлами	27
src/ main.cpp	
Главный модуль клиентского приложения	28
src/ test_main.cpp	
Модульные тесты для клиентского приложения	30
src/ user_interface.cpp	
Реализация класса пользовательского интерфейса	33
src/ user_interface.h	
Заголовочный файл класса пользовательского интерфейса	33

Глава 5

Class Documentation

5.1 AuthManager Class Reference

Статический класс для криптографических операций

```
#include <auth_manager.h>
```

Static Public Member Functions

- static std::string [generateSalt](#) ()
- static std::string [computeHash](#) (const std::string &salt, const std::string &password)

Private Member Functions

- [AuthManager](#) ()=delete
- [~AuthManager](#) ()=delete

5.1.1 Detailed Description

Статический класс для криптографических операций

Definition at line 15 of file auth_manager.h.

5.1.2 Constructor & Destructor Documentation

5.1.2.1 AuthManager()

AuthManager::AuthManager () [private], [delete]

5.1.2.2 ~AuthManager()

AuthManager::~AuthManager () [private], [delete]

5.1.3 Member Function Documentation

5.1.3.1 computeHash()

```
std::string AuthManager::computeHash (
    const std::string & salt,
    const std::string & password ) [static]
```

Definition at line 26 of file auth_manager.cpp.

5.1.3.2 generateSalt()

```
std::string AuthManager::generateSalt ( ) [static]
```

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 16 of file auth_manager.cpp.

The documentation for this class was generated from the following files:

- [src/auth_manager.h](#)
- [src/auth_manager.cpp](#)

5.2 ClientParams Struct Reference

Структура для хранения параметров клиента

```
#include <user_interface.h>
```

Public Attributes

- std::string [server_address](#)
Адрес сервера
- uint16_t [port](#) = 33333
Порт сервера
- std::string [input_file](#)
Входной файл
- std::string [output_file](#)
Выходной файл
- std::string [config_file](#) = "~/config/vclient.conf"
Файл конфигурации

5.2.1 Detailed Description

Структура для хранения параметров клиента

Definition at line 14 of file user_interface.h.

5.2.2 Member Data Documentation

5.2.2.1 config_file

```
std::string ClientParams::config_file = "~/config/vclient.conf"
```

Файл конфигурации

Definition at line 19 of file user_interface.h.

5.2.2.2 input_file

```
std::string ClientParams::input_file
```

Входной файл

Definition at line 17 of file user_interface.h.

5.2.2.3 output_file

```
std::string ClientParams::output_file
```

Выходной файл

Definition at line 18 of file user_interface.h.

5.2.2.4 port

```
uint16_t ClientParams::port = 33333
```

Порт сервера

Definition at line 16 of file user_interface.h.

5.2.2.5 server_address

std::string ClientParams::server_address

Адрес сервера

Definition at line 15 of file user_interface.h.

The documentation for this struct was generated from the following file:

- src/[user_interface.h](#)

5.3 ConnectionManager Class Reference

Класс для управления TCP-соединением с сервером

```
#include <connection_manager.h>
```

Public Member Functions

- [ConnectionManager](#) (const std::string &server_addr, uint16_t port)
- virtual [~ConnectionManager](#) ()
- void [authenticate](#) (const std::string &login, const std::string &password)
- std::vector< int32_t > [processVectors](#) (const std::vector< std::vector< int32_t >> &vectors)

Protected Member Functions

- virtual void [sendData](#) (const void *data, size_t size)
- virtual void [receiveData](#) (void *data, size_t size)
- virtual void [sendString](#) (const std::string &str)
- virtual std::string [receiveString](#) (size_t max_length=1024)

Protected Attributes

- int [socket_](#)
- std::string [server_addr_](#)
- uint16_t [port_](#)

5.3.1 Detailed Description

Класс для управления TCP-соединением с сервером

Definition at line 21 of file connection_manager.h.

5.3.2 Constructor & Destructor Documentation

5.3.2.1 ConnectionManager()

```
ConnectionManager::ConnectionManager (
    const std::string & server_addr,
    uint16_t port )
```

Definition at line 12 of file connection_manager.cpp.

5.3.2.2 ~ConnectionManager()

```
ConnectionManager::~~ConnectionManager ( ) [virtual]
```

Definition at line 49 of file connection_manager.cpp.

5.3.3 Member Function Documentation

5.3.3.1 authenticate()

```
void ConnectionManager::authenticate (
    const std::string & login,
    const std::string & password )
```

Definition at line 55 of file connection_manager.cpp.

5.3.3.2 processVectors()

```
std::vector< int32_t > ConnectionManager::processVectors (
    const std::vector< std::vector< int32_t >> & vectors )
```

Definition at line 73 of file connection_manager.cpp.

5.3.3.3 receiveData()

```
void ConnectionManager::receiveData (
    void * data,
    size_t size ) [protected], [virtual]
```

Definition at line 111 of file connection_manager.cpp.

5.3.3.4 receiveString()

```
std::string ConnectionManager::receiveString (  
    size_t max_length = 1024 )    [protected], [virtual]
```

Definition at line 131 of file connection_manager.cpp.

5.3.3.5 sendData()

```
void ConnectionManager::sendData (  
    const void * data,  
    size_t size )    [protected], [virtual]
```

Definition at line 98 of file connection_manager.cpp.

5.3.3.6 sendString()

```
void ConnectionManager::sendString (  
    const std::string & str )    [protected], [virtual]
```

Definition at line 127 of file connection_manager.cpp.

5.3.4 Member Data Documentation

5.3.4.1 port_

```
uint16_t ConnectionManager::port_    [protected]
```

Definition at line 32 of file connection_manager.h.

5.3.4.2 server_addr_

```
std::string ConnectionManager::server_addr_    [protected]
```

Definition at line 31 of file connection_manager.h.

5.3.4.3 socket_

int ConnectionManager::socket_ [protected]

Definition at line 30 of file connection_manager.h.

The documentation for this class was generated from the following files:

- src/connection_manager.h
- src/connection_manager.cpp

5.4 FileHandler Class Reference

Статический класс для операций с файлами

```
#include <file_handler.h>
```

Static Public Member Functions

- static std::vector< std::vector< int32_t > > [readVectors](#) (const std::string &filename)
- static void [saveResults](#) (const std::string &filename, const std::vector< int32_t > &results)
- static std::pair< std::string, std::string > [readConfig](#) (const std::string &filename)

Private Member Functions

- [FileHandler](#) ()=delete
- [~FileHandler](#) ()=delete
- [FileHandler](#) (const [FileHandler](#) &)=delete
- [FileHandler](#) & [operator=](#) (const [FileHandler](#) &)=delete

5.4.1 Detailed Description

Статический класс для операций с файлами

Definition at line 18 of file file_handler.h.

5.4.2 Constructor & Destructor Documentation

5.4.2.1 FileHandler() [1/2]

FileHandler::FileHandler () [private], [delete]

5.4.2.2 ~FileHandler()

FileHandler::~~FileHandler () [private], [delete]

5.4.2.3 FileHandler() [2/2]

FileHandler::FileHandler (
const [FileHandler](#) &) [private], [delete]

5.4.3 Member Function Documentation

5.4.3.1 operator=()

[FileHandler](#)& FileHandler::operator= (
const [FileHandler](#) &) [private], [delete]

5.4.3.2 readConfig()

std::pair< std::string, std::string > FileHandler::readConfig (
const std::string & filename) [static]

Definition at line 70 of file file_handler.cpp.

5.4.3.3 readVectors()

std::vector< std::vector< int32_t > > FileHandler::readVectors (
const std::string & filename) [static]

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 11 of file file_handler.cpp.

5.4.3.4 saveResults()

```
void FileHandler::saveResults (
    const std::string & filename,
    const std::vector< int32_t > & results ) [static]
```

Definition at line 50 of file file_handler.cpp.

The documentation for this class was generated from the following files:

- [src/file_handler.h](#)
- [src/file_handler.cpp](#)

5.5 TestFileFixture Struct Reference

Фикстура для тестирования работы с файлами

Public Member Functions

- [TestFileFixture \(\)](#)
Конструктор фикстуры
- [~TestFileFixture \(\)](#)
Деструктор фикстуры

Public Attributes

- std::string [test_filename](#)
Имя тестового входного файла
- std::string [test_output](#)
Имя тестового выходного файла

5.5.1 Detailed Description

Фикстура для тестирования работы с файлами

Создает временные файлы для тестирования и автоматически удаляет их после завершения тестов

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 59 of file test_main.cpp.

5.5.2 Constructor & Destructor Documentation

5.5.2.1 TestFileFixture()

TestFileFixture::TestFileFixture () [inline]

Конструктор фикстуры

Создает тестовый файл с векторами

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 67 of file test_main.cpp.

5.5.2.2 ~TestFileFixture()

TestFileFixture::~~TestFileFixture () [inline]

Деструктор фикстуры

Удаляет все созданные временные файлы

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 82 of file test_main.cpp.

5.5.3 Member Data Documentation

5.5.3.1 test_filename

std::string TestFileFixture::test_filename

Имя тестового входного файла

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 60 of file test_main.cpp.

5.5.3.2 test_output

`std::string TestFileFixture::test_output`

Имя тестового выходного файла

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 61 of file `test_main.cpp`.

The documentation for this struct was generated from the following file:

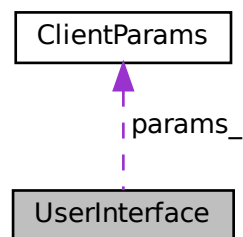
- [src/test_main.cpp](#)

5.6 UserInterface Class Reference

Класс для обработки параметров командной строки

```
#include <user_interface.h>
```

Collaboration diagram for UserInterface:



Public Member Functions

- [UserInterface](#) ()
- [bool parseArguments](#) (int argc, char **argv)
- [ClientParams getParams](#) () const
- `std::string` [getHelp](#) () const

Private Attributes

- `boost::program_options::options_description` [desc_](#)
- [ClientParams](#) [params_](#)

5.6.1 Detailed Description

Класс для обработки параметров командной строки

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 26 of file user_interface.h.

5.6.2 Constructor & Destructor Documentation

5.6.2.1 UserInterface()

UserInterface::UserInterface ()

Definition at line 10 of file user_interface.cpp.

5.6.3 Member Function Documentation

5.6.3.1 getHelp()

std::string UserInterface::getHelp () const

Definition at line 55 of file user_interface.cpp.

5.6.3.2 getParams()

[ClientParams](#) UserInterface::getParams () const [inline]

Definition at line 30 of file user_interface.h.

5.6.3.3 parseArguments()

```
bool UserInterface::parseArguments (
    int argc,
    char ** argv )
```

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 20 of file user_interface.cpp.

5.6.4 Member Data Documentation

5.6.4.1 desc_

`boost::program_options::options_description UserInterface::desc_ [private]`

Definition at line 34 of file `user_interface.h`.

5.6.4.2 params_

`ClientParams UserInterface::params_ [private]`

Definition at line 35 of file `user_interface.h`.

The documentation for this class was generated from the following files:

- `src/user_interface.h`
- `src/user_interface.cpp`

Глава 6

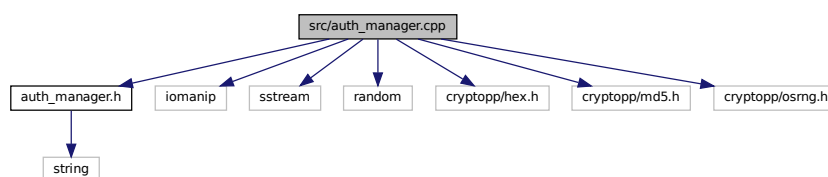
File Documentation

6.1 README.md File Reference

6.2 src/auth_manager.cpp File Reference

Реализация класса аутентификации

```
#include "auth_manager.h"  
#include <iomanip>  
#include <sstream>  
#include <random>  
#include <cryptopp/hex.h>  
#include <cryptopp/md5.h>  
#include <cryptopp/osrng.h>  
Include dependency graph for auth_manager.cpp:
```



6.2.1 Detailed Description

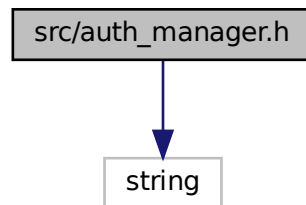
Реализация класса аутентификации

6.3 src/auth_manager.h File Reference

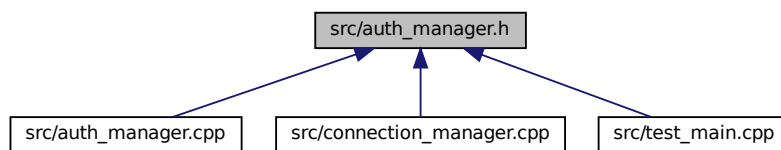
Заголовочный файл класса аутентификации

```
#include <string>
```

Include dependency graph for auth_manager.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [AuthManager](#)
Статический класс для криптографических операций

Macros

- `#define` [CRYPTOPP_ENABLE_NAMESPACE_WEAK](#) 1

6.3.1 Detailed Description

Заголовочный файл класса аутентификации

6.3.2 Macro Definition Documentation

6.3.2.1 CRYPTOPP_ENABLE_NAMESPACE_WEAK

```
#define CRYPTOPP_ENABLE_NAMESPACE_WEAK 1
```

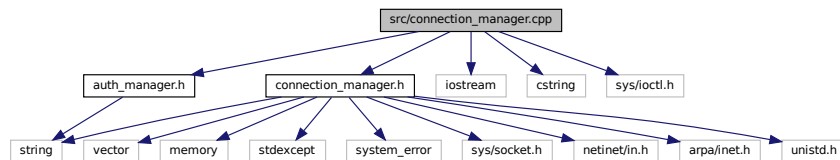
Definition at line 9 of file auth_manager.h.

6.4 src/connection_manager.cpp File Reference

Реализация класса управления соединением

```
#include "connection_manager.h"
#include "auth_manager.h"
#include <iostream>
#include <cstring>
#include <sys/ioctl.h>
```

Include dependency graph for connection_manager.cpp:



6.4.1 Detailed Description

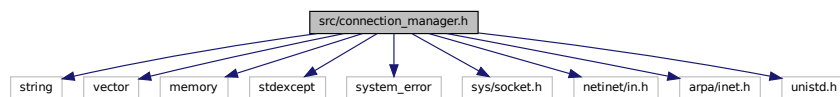
Реализация класса управления соединением

6.5 src/connection_manager.h File Reference

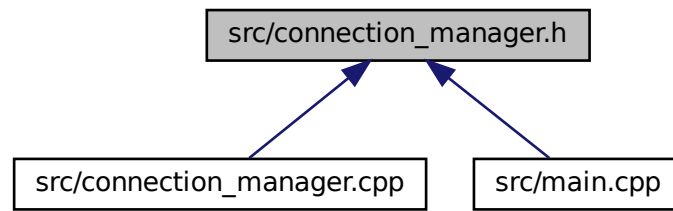
Заголовочный файл класса управления соединением

```
#include <string>
#include <vector>
#include <memory>
#include <stdexcept>
#include <system_error>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
```

Include dependency graph for connection_manager.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [ConnectionManager](#)
Класс для управления TCP-соединением с сервером

6.5.1 Detailed Description

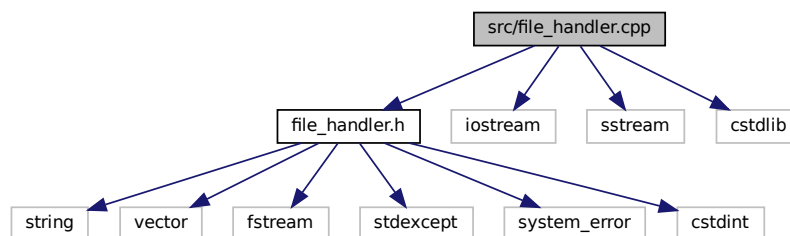
Заголовочный файл класса управления соединением

6.6 src/file_handler.cpp File Reference

Реализация класса работы с файлами

```
#include "file_handler.h"
#include <iostream>
#include <sstream>
#include <cstdlib>
```

Include dependency graph for file_handler.cpp:



6.6.1 Detailed Description

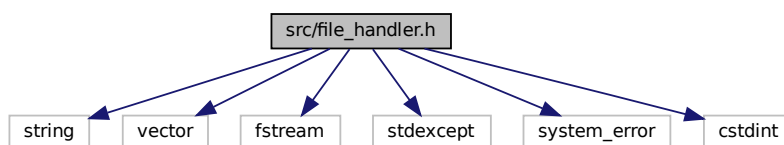
Реализация класса работы с файлами

6.7 src/file_handler.h File Reference

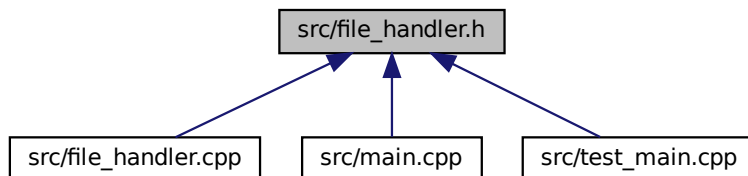
Заголовочный файл класса работы с файлами

```
#include <string>
#include <vector>
#include <fstream>
#include <stdexcept>
#include <system_error>
#include <cstdint>
```

Include dependency graph for file_handler.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [FileHandler](#)
Статический класс для операций с файлами

6.7.1 Detailed Description

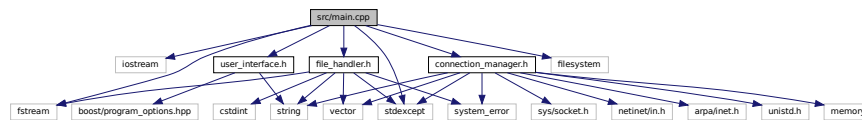
Заголовочный файл класса работы с файлами

6.8 src/main.cpp File Reference

Главный модуль клиентского приложения

```
#include <iostream>
#include <stdexcept>
#include "user_interface.h"
#include "file_handler.h"
#include "connection_manager.h"
#include <fstream>
#include <filesystem>
```

Include dependency graph for main.cpp:



Functions

- void [createDemoInputFile](#) (const std::string &filename)
Создает демонстрационный входной файл
- void [createConfigFile](#) (const std::string &filename)
Создает конфигурационный файл
- void [ensureFilesExist](#) (const std::string &input_file, const std::string &config_file)
Проверяет и создает отсутствующие файлы
- int [main](#) (int argc, char **argv)
Точка входа в программу

6.8.1 Detailed Description

Главный модуль клиентского приложения

6.8.2 Function Documentation

6.8.2.1 createConfigFile()

```
void createConfigFile (
    const std::string & filename )
```

Создает конфигурационный файл

Parameters

filename	Имя конфигурационного файла
----------	-----------------------------

Definition at line 40 of file main.cpp.

6.8.2.2 createDemoInputFile()

```
void createDemoInputFile (  
    const std::string & filename )
```

Создает демонстрационный входной файл

Parameters

filename	Имя файла для создания
----------	------------------------

Definition at line 18 of file main.cpp.

6.8.2.3 ensureFilesExist()

```
void ensureFilesExist (  
    const std::string & input_file,  
    const std::string & config_file )
```

Проверяет и создает отсутствующие файлы

Parameters

input_file	Входной файл
config_file	Конфигурационный файл

Definition at line 67 of file main.cpp.

6.8.2.4 main()

```
int main (  
    int argc,  
    char ** argv )
```

Точка входа в программу

Parameters

argc	Количество аргументов
argv	Массив аргументов

Returns

0 при успехе, 1 при ошибке

Examples

[/home/stud/client_project/src/test_main.cpp](#).

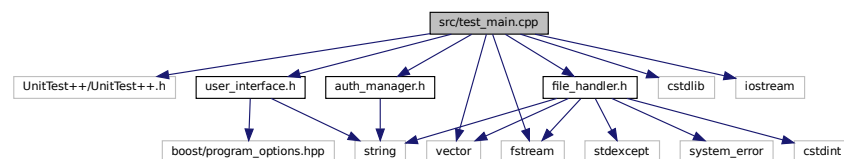
Definition at line 93 of file main.cpp.

6.9 src/test_main.cpp File Reference

Модульные тесты для клиентского приложения

```
#include <UnitTest++/UnitTest++.h>
#include "user_interface.h"
#include "file_handler.h"
#include "auth_manager.h"
#include <fstream>
#include <cstdlib>
#include <vector>
#include <iostream>
```

Include dependency graph for test_main.cpp:



Classes

- struct [TestFixture](#)
Фикстура для тестирования работы с файлами

Functions

- [SUITE](#) (UserInterfaceTest)
- [SUITE](#) (FileHandlerTest)
- [SUITE](#) (AuthManagerTest)
- int [main](#) ()
Главная функция тестов

6.9.1 Detailed Description

Модульные тесты для клиентского приложения

Author

Пресняков А.А.

Version

1.0

Date

28.10.2025

Copyright

ИБСТ ПГУ

Содержит модульные тесты для всех компонентов системы:

- [UserInterface](#) (пользовательский интерфейс)
- [FileHandler](#) (работа с файлами)
- [AuthManager](#) (аутентификация)
- Интеграционные тесты

Note

Использует фреймворк `UnitTest++`

Все тесты выполняются автоматически при запуске

6.9.2 Function Documentation

6.9.2.1 `main()`

`int main ( )`

Главная функция тестов

Returns

Количество неудачных тестов (0 - все тесты прошли)

Запускает все тесты и выводит результаты

Definition at line 134 of file `test_main.cpp`.

6.9.2.2 SUITE() [1/3]

SUITE (
 AuthManagerTest)

[Test](#) GenerateSaltLength

Тест длины генерируемой соли

Definition at line 114 of file test_main.cpp.

6.9.2.3 SUITE() [2/3]

SUITE (
 FileHandlerTest)

[Test](#) ReadVectorsValid

Тест чтения корректного файла с векторами

Definition at line 92 of file test_main.cpp.

6.9.2.4 SUITE() [3/3]

SUITE (
 UserInterfaceTest)

[Test](#) HelpShort

Тест короткой формы справки (-h)

Examples

[/home/stud/client_project/src/test_main.cpp](#).

Definition at line 36 of file test_main.cpp.

6.10 src/user_interface.cpp File Reference

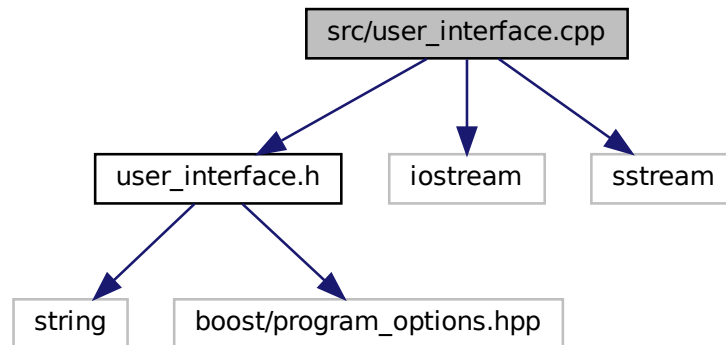
Реализация класса пользовательского интерфейса

```
#include "user_interface.h"
```

```
#include <iostream>
```

```
#include <sstream>
```

Include dependency graph for user_interface.cpp:



6.10.1 Detailed Description

Реализация класса пользовательского интерфейса

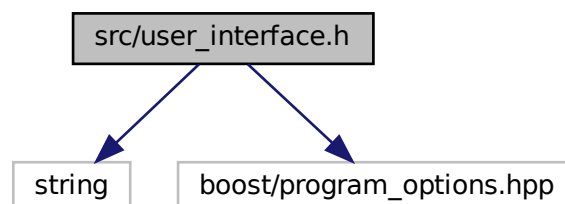
6.11 src/user_interface.h File Reference

Заголовочный файл класса пользовательского интерфейса

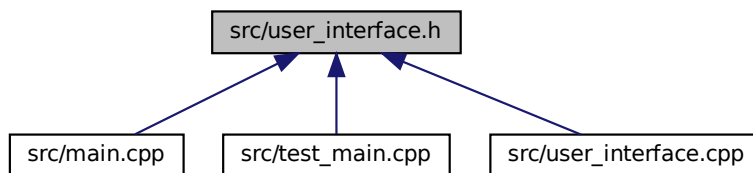
```
#include <string>
```

```
#include <boost/program_options.hpp>
```

Include dependency graph for user_interface.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [ClientParams](#)
Структура для хранения параметров клиента
- class [UserInterface](#)
Класс для обработки параметров командной строки

6.11.1 Detailed Description

Заголовочный файл класса пользовательского интерфейса

Глава 7

Example Documentation

7.1 /home/stud/client_project/src/test_main.cpp

```
// Сборка и запуск тестов
g++ -o test_client test_main.cpp user_interface.cpp file_handler.cpp \    auth_manager.cpp -lUnitTest++ -lcryptopp
./test_client
#include <UnitTest++/UnitTest++.h>
#include "user_interface.h"
#include "file_handler.h"
#include "auth_manager.h"
#include <fstream>
#include <cstdlib>
#include <vector>
#include <iostream>
// Тестирование пользовательского интерфейса
SUITE(UserInterfaceTest)
{
    TEST(HelpShort) {
        UserInterface ui;
        const char* argv[] = {"test", "-h", nullptr};
        int argc = sizeof(argv) / sizeof(argv[0]) - 1;
        CHECK(!ui.parseArguments(argc, const_cast<char**>(argv)));
    }

    // ... остальные тесты UserInterface
}
// Фикстура для тестирования файлов
struct TestFileFixture {
    std::string test_filename;
    std::string test_output;

    TestFileFixture() : test_filename("test_input.txt"), test_output("test_output.bin") {
        // Создаем тестовый файл
        std::ofstream file(test_filename);
        file << "2\n";           // количество векторов
        file << "3\n";           // размер первого вектора
        file << "1 2 3\n";       // значения первого вектора
        file << "2\n";           // размер второго вектора
        file << "10 20\n";        // значения второго вектора
        file.close();
    }

    ~TestFileFixture() {
        std::remove(test_filename.c_str());
        std::remove(test_output.c_str());
        std::remove("test_config.conf");
        std::remove("empty_file.txt");
        std::remove("invalid_file.txt");
    }
};
// Тестирование работы с файлами
SUITE(FileHandlerTest)
{
    TEST_FIXTURE(TestFileFixture, ReadVectorsValid) {
        auto vectors = FileHandler::readVectors("test_input.txt");
        CHECK_EQUAL(2, vectors.size());
        CHECK_EQUAL(3, vectors[0].size());
        CHECK_EQUAL(2, vectors[1].size());
        CHECK_EQUAL(1, vectors[0][0]);
        CHECK_EQUAL(2, vectors[0][1]);
    }
}
```

```
        CHECK_EQUAL(3, vectors[0][2]);
        CHECK_EQUAL(10, vectors[1][0]);
        CHECK_EQUAL(20, vectors[1][1]);
    }

    // ... остальные тесты FileHandler
}
// Тестирование аутентификации
SUITE(AuthManagerTest)
{
    TEST(GenerateSaltLength) {
        std::string salt = AuthManager::generateSalt();
        CHECK_EQUAL(16, salt.length()); // 64 бита = 16 hex цифр
    }

    // ... остальные тесты AuthManager
}
int main() {
    std::cout << "=== Запуск модульных тестов клиента ===" << std::endl;
    std::cout << "Тестирование всех компонентов системы..." << std::endl;

    // Запуск всех тестов
    int result = UnitTest::RunAllTests();

    if (result == 0) {
        std::cout << "=== ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО ===" << std::endl;
    } else {
        std::cout << "=== НЕКОТОРЫЕ ТЕСТЫ НЕ ПРОЙДЕНЫ ===" << std::endl;
    }

    return result;
}
```